

PRINCIPLES OF SOFT COMPUTING LAB ASSIGNMENTS REPORT (CSE 412L)

BY

PERUMALLA DHARAN (AP21110010201)



**SRM University - AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh - 522240
Nov, 2024**

SOFT COMPUTING

ASSIGNMENT -1

Perumalla Dharan

AP21110010201

1. Write a Python program to check whether a student has failed or not: The input data is students' semester marks, which is supplied as an external file.

```
import numpy as np
import pandas as pd
data=pd.read_csv('Student_data.csv')
marks=data[['c1','c2','c3','c4','c5','c6']].values
for X in marks:
    flag=0
    for i in X:
        if i<50:
            flag=1
            break
    print(X,end=' --> ')
    if(flag):
        print('Fail')
    else:
        print('Pass')
```

Output-

```
[58 17 74 58 11 63] --> Fail
[56 74 99 26 10 3] --> Fail
[41 16 32 26 94 47] --> Fail
[62 32 70 25 49 37] --> Fail
[60 76 20 4 41 58] --> Fail
[42 66 15 84 38 15] --> Fail
[64 79 58 68 65 66] --> Pass
[43 93 82 92 15 73] --> Fail
[56 57 10 21 94 27] --> Fail
[95 87 99 93 60 33] --> Fail
[33 34 90 39 29 50] --> Fail
[ 66 86 35 56 79 100] --> Fail
[66 21 46 34 38 57] --> Fail
[81 81 1 18 78 67] --> Fail
[30 77 64 13 36 97] --> Fail
```

SOFT COMPUTING

ASSIGNMENT -2

Perumalla Dharan

AP21110010201

1.

Write a Python program to implement to predict whether a student has failed or not using the following classification algorithms:

The input data is students' semester marks, which is supplied as an external file.

DECISION TREE

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv('training_dataset_students(1000).csv')
y = data.iloc[:, -1].values
X = data.iloc[:, 1:-1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3, random_state=42)

dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
print("Decision Tree:")
print("Accuracy:", round(accuracy_score(y_test,
y_pred_dt)*100,2), "%")
```

```
data = pd.read_csv('students_testing.csv')
new_data = data.iloc[:, 1:-1].values

print(new_data)

pred_dt_new = dt_model.predict(new_data)
print("\nDecision Tree Prediction for New Data:", pred_dt_new)
```

Decision Tree:
Accuracy: 99.67 %

```
[[32 21 93 78 93 53]
 [81 95 68 75 82 83]
 [ 8  5 31 49 21 17]
 [80 47 25 37 27 56]
 [53 68 95 50 68 54]
 [73 63 66 56 34 76]
 [ 0 37 12 41 19 16]
 [78 17 47 86 79 26]
 [71 65 64 42  6 85]
 [21 49 67 93 80 32]
 [50 55 56 56 58 59]]
```

```
Decision Tree Prediction for New Data: [0 1 0 0 1 1 0 0 0 0 1]
```

2.

Write a Python program to implement to predict whether a student has failed or not using the following classification algorithms..

The input data is students' semester marks, which is supplied as an external file.

KNN

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv('training_dataset_students(1000).csv')
y = data.iloc[:, -1].values
X = data.iloc[:, 1:-1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=42)

knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
print("\nKNN:")
print("Accuracy:", round(accuracy_score(y_test,
    y_pred_knn)*100,2), "%")

data = pd.read_csv('students_testing.csv')
new_data = data.iloc[:, 1:-1].values

print(new_data)
```

```
pred_knn_new = knn_model.predict(new_data)
print("KNN Prediction for New Data:", pred_knn_new)
```

KNN:

Accuracy: 93.0 %

```
[[32 21 93 78 93 53]
 [81 95 68 75 82 83]
 [ 8  5 31 49 21 17]
 [80 47 25 37 27 56]
 [53 68 95 50 68 54]
 [73 63 66 56 34 76]
 [ 0 37 12 41 19 16]
 [78 17 47 86 79 26]
 [71 65 64 42  6 85]
 [21 49 67 93 80 32]
 [50 55 56 56 58 59]]
KNN Prediction for New Data: [0 1 0 0 1 1 0 0 0 0 1]
```

3.

Write a Python program to implement to predict whether a student has failed or not using the following classification algorithms:

The input data is students' semester marks, which is supplied as an external file.

SVM

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv('training_dataset_students(1000).csv')
y = data.iloc[:, -1].values
X = data.iloc[:, 1:-1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=42)

svm_model = SVC()
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
print("\nSVM:")
print("Accuracy:", round(accuracy_score(y_test,
    y_pred_svm)*100,2), "%")

data = pd.read_csv('students_testing.csv')
new_data = data.iloc[:, 1:-1].values

print(new_data)

pred_svm_new = svm_model.predict(new_data)
print("SVM Prediction for New Data:", pred_svm_new)
```

SVM:

Accuracy: 97.0 %

```
[[32 21 93 78 93 53]
 [81 95 68 75 82 83]
 [ 8  5 31 49 21 17]
 [80 47 25 37 27 56]
 [53 68 95 50 68 54]
 [73 63 66 56 34 76]
 [ 0 37 12 41 19 16]
 [78 17 47 86 79 26]
 [71 65 64 42  6 85]
 [21 49 67 93 80 32]
 [50 55 56 56 58 59]]
SVM Prediction for New Data: [0 1 0 0 1 1 0 0 0 0 1]
```

4.

Write a Python program to implement to predict whether a student has failed or not using the following classification algorithms:.

The input data is students' semester marks, which is supplied as an external file.

Perceptron (Use built-in function)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report
```

```
data = pd.read_csv('training_dataset_students(1000).csv')
y = data.iloc[:, -1].values
X = data.iloc[:, 1:-1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=42)

perceptron_model = Perceptron()
perceptron_model.fit(X_train, y_train)
y_pred_perceptron = perceptron_model.predict(X_test)
print("\nPerceptron:")
print("Accuracy:", round(accuracy_score(y_test,
    y_pred_perceptron)*100,2), "%")

data = pd.read_csv('students_testing.csv')
new_data = data.iloc[:, 1:-1].values

print(new_data)

pred_perceptron_new = perceptron_model.predict(new_data)
print("Perceptron Prediction for New Data:", pred_perceptron_new)
```

Perceptron:
Accuracy: 52.67 %

```
[[32 21 93 78 93 53]
 [81 95 68 75 82 83]
 [ 8  5 31 49 21 17]
 [80 47 25 37 27 56]
 [53 68 95 50 68 54]
 [73 63 66 56 34 76]
 [ 0 37 12 41 19 16]
 [78 17 47 86 79 26]
 [71 65 64 42  6 85]
 [21 49 67 93 80 32]
 [50 55 56 56 58 59]]
```

```
Perceptron Prediction for New Data: [1 1 0 1 1 1 1 1 1 1 1]
```

SOFT COMPUTING

ASSIGNMENT -3

Perumalla Dharan

AP21110010201

Write a Python program to implement a Perceptron. The input is your semester marks. Do not use the built-in functions of Perceptron.

Note: Upload the source file of the Python program and a Word document file that contains the Python program along with the results.

```
import numpy as np
import pandas as pd

class Perceptron:
    def __init__(self, input_size, learning_rate=1, iterations=10):
        self.weights = np.zeros(input_size + 1)
        self.learning_rate = learning_rate
        self.iterations = iterations

    def activation_function(self, x):
        return 1 if x >= 0 else 0

    def predict(self, x):
        z = np.dot(x, self.weights[1:]) + self.weights[0]
        return self.activation_function(z)

    def train(self, X, y):
        for _ in range(self.iterations):
            print(f"\nIteration {_+1}:")
            for xi, target in zip(X, y):
                prediction = self.predict(xi)
                print(f"Sample: {xi}")
```

```

        print(f"Prediction: {prediction}, Target:
{target}")

        print(f"Current weights: {self.weights}")
        if prediction != target:
            self.weights[1:] += self.learning_rate *
(target-prediction) * xi
            self.weights[0] += self.learning_rate * (target
)

            print(f"Updated weights: {self.weights}")
            print(f"Updated bias: {self.weights[0]}")
        else:
            print("No Update in weights required")
        print("\n")

    def evaluate(self, X, y):
        predictions = [self.predict(xi) for xi in X]
        accuracy = np.mean(predictions == y)
        return accuracy

df = pd.read_csv('random_data.csv')

X = df[['c1', 'c2', 'c3', 'c4', 'c5', 'c6']].values
y = df['result'].values

perceptron = Perceptron(input_size=X.shape[1], learning_rate=1,
iterations=10)

perceptron.train(X, y)
accuracy = perceptron.evaluate(X, y)
print(f"Training accuracy: {accuracy * 100:.2f}%")

new_marks = np.array([7,0,2,3,4,5])
prediction = perceptron.predict(new_marks)
print(f"Prediction for new marks (Pass=1, Fail=0): {prediction}")

```

Training accuracy: 50.00%

Prediction for new marks (Pass=1, Fail=0): 1

SOFT COMPUTING

ASSIGNMENT -4

Perumalla Dharan

AP21110010201

Write a Python program to implement a Multi-Layer Perceptron (MLP) on logical XOR function. Take the binary inputs and outputs.

Note: Upload the source file of the Python program and a Word document file that contains the Python program along with the results.

```
import numpy as np

class MLP:
    def __init__(self, learning_rate=0.1, iterations=10000):
        self.learning_rate = learning_rate
        self.iterations = iterations

        self.weights_input_hidden = np.random.uniform(-0.5, 0.5,
(2, 2))
        self.bias_hidden = np.zeros(2)
        self.weights_hidden_output = np.random.uniform(-0.5, 0.5,
(2, 1))
        self.bias_output = np.zeros(1)

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def predict(self, X):
        hidden_input = np.dot(X, self.weights_input_hidden) +
self.bias_hidden
        hidden_output = self.sigmoid(hidden_input)
```

```

        final_input = np.dot(hidden_output,
self.weights_hidden_output) + self.bias_output
        final_output = self.sigmoid(final_input)
        return np.round(final_output)

    def train(self, X, y):
        for epoch in range(self.iterations):
            for i in range(len(X)):

                hidden_input = np.dot(X[i],
self.weights_input_hidden) + self.bias_hidden
                hidden_output = self.sigmoid(hidden_input)

                final_input = np.dot(hidden_output,
self.weights_hidden_output) + self.bias_output
                final_output = self.sigmoid(final_input)

                y_pred = np.round(final_output)

                if y_pred != y[i]:

                    output_error = y[i] - final_output
                    hidden_error =
output_error.dot(self.weights_hidden_output.T)

                    self.weights_hidden_output +=
self.learning_rate * np.outer(hidden_output, output_error)
                    self.bias_output += self.learning_rate *
output_error

                    self.weights_input_hidden += self.learning_rate
* np.outer(X[i], hidden_error)
                    self.bias_hidden += self.learning_rate *
hidden_error

```

```
def print_weights(self):
    print("Input-Hidden Weights:\n", self.weights_input_hidden)
    print("Hidden-Output Weights:\n",
self.weights_hidden_output)

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0], [1], [1], [0]])

mlp = MLP(learning_rate=0.1, iterations=10000)

mlp.train(X, y)

mlp.print_weights()

predictions = mlp.predict(X)
print("Predictions after training:")
print(np.round(predictions))
```

Output -

Input-Hidden Weights:

```
[[ -24.55377215   2.08147086]  
[  0.8444042   -1.65447977]]
```

Hidden-Output Weights:

```
[[0.37799175]  
[0.25812419]]
```

Predictions after training:

```
[[0.]  
[1.]  
[1.]  
[0.]]
```

SOFT COMPUTING

ASSIGNMENT -5

Perumalla Dharan

AP21110010201

Write a Python program to implement a MADALINE on logical XOR function. Take the binary inputs and outputs.

Note: Upload the source file of the Python program and a Word document file that contains the Python program along with the results.

```
import numpy as np

class MADALINE:
    def __init__(self, learning_rate=0.1, iterations=10000):
        self.learning_rate = learning_rate
        self.iterations = iterations

        self.w = np.random.uniform(-0.5, 1, (2, 2))
        self.b1 = np.random.uniform(-0.5, 1, 2)

        self.v = np.random.uniform(-0.5, 1, (2, 1))
        self.b2 = np.random.uniform(-0.5, 1, 1)

    def fx(self, x):
        return np.where(x >= 0, 1, 0)

    def predict(self, X):

        z_in = np.dot(X, self.w) + self.b1
        z_out = self.fx(z_in)
```

```

        y_in = np.dot(z_out, self.v) + self.b2
        y_out = self.fx(y_in)

    return y_out

def train(self, X, y):
    for epoch in range(self.iterations):
        for i in range(len(X)):

            z_in = np.dot(X[i], self.w) + self.b1
            z_out = self.fx(z_in)

            y_in = np.dot(z_out, self.v) + self.b2
            y_out = self.fx(y_in)

            if y_out != y[i]:
                if y[i] == -1:
                    for j in range(len(z_in)):
                        if z_in[j] > 0:

                            self.w[:, j] += (self.learning_rate
* (X[i]) * (y[i]-z_in[j].item()))
                            self.b1[j] += (self.learning_rate *
(y[i]-z_in[j].item()))

                elif y[i] == 1:

                    closest_to_zero_index =
np.argmin(np.abs(z_in))

                    self.w[:, closest_to_zero_index] +=
(self.learning_rate * X[i] * (y[i]-z_in[j].item()))
                    self.b1[closest_to_zero_index] +=
(self.learning_rate * (y[i]-z_in[closest_to_zero_index].item()))

```

```
# Input Data for training (XOR logic)
X = np.array([[0,0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Input Data for testing (same as XOR)
Z = np.array([[0,0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0], [1], [1], [0]])

madaline = MADALINE(learning_rate=0.1, iterations=10000)

madaline.train(X, y)

predictions = madaline.predict(Z)

# Display results
print("Testing Data:")
print(Z)
print("Predictions after training:")
print(predictions)
```

Testing Data:

```
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
```

Predictions after training:

```
[[1]
 [1]
 [1]
 [1]]
```

SOFT COMPUTING

ASSIGNMENT -6

Perumalla Dharan

AP21110010201

1. Write a Python program to implement a Multi-Layer Perceptron (MLP) for classifying whether a student passes or fails using a dataset of students' course marks.

```
import numpy as np
import pandas as pd

train_path = r"E:\SRM\Soft Computing\Lab 6\students_testing.csv"
train_df = pd.read_csv(train_path)
test_path = r"E:\SRM\Soft Computing\Lab 6\training_dataset_students(1000).csv"
test_df = pd.read_csv(test_path)

X_train = train_df[['c1', 'c2', 'c3', 'c4', 'c5', 'c6']].values
y_train = train_df[['result']].values

X_test = test_df[['c1', 'c2', 'c3', 'c4', 'c5', 'c6']].values
y_test = test_df[['result']].values

class MLP:
    def __init__(self, input_size, hidden_size, learning_rate=0.1, iterations=100):
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.weights_input_hidden = np.random.uniform(-0.5, 0.5, (input_size, hidden_size))
        self.bias_hidden = np.zeros(hidden_size)
        self.weights_hidden_output = np.random.uniform(-0.5, 0.5, (hidden_size, 1))
```

```

        self.bias_output = np.zeros(1)

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def predict(self, X):
        hidden_input = np.dot(X, self.weights_input_hidden) +
self.bias_hidden
        hidden_output = self.sigmoid(hidden_input)

        final_input = np.dot(hidden_output,
self.weights_hidden_output) + self.bias_output
        final_output = self.sigmoid(final_input)
        return np.round(final_output)

    def train(self, X, y):
        for epoch in range(self.iterations):
            for i in range(len(X)):

                hidden_input = np.dot(X[i],
self.weights_input_hidden) + self.bias_hidden
                hidden_output = self.sigmoid(hidden_input)

                final_input = np.dot(hidden_output,
self.weights_hidden_output) + self.bias_output
                final_output = self.sigmoid(final_input)
                y_pred = np.round(final_output)

                output_error = y[i] - final_output

                hidden_error =
output_error.dot(self.weights_hidden_output.T) * hidden_output *
(1 - hidden_output)

                self.weights_hidden_output += self.learning_rate *
np.outer(hidden_output, output_error)

```

```

        self.bias_output += self.learning_rate *
output_error

        self.weights_input_hidden += self.learning_rate *
np.outer(X[i], hidden_error)

        self.bias_hidden += self.learning_rate *
hidden_error

    def print_weights(self):
        print("Input-Hidden Weights:\n", self.weights_input_hidden)
        print("Hidden-Output Weights:\n",
self.weights_hidden_output)

input_size = X_train.shape[1]
hidden_size = 4

mlp = MLP(input_size=input_size, hidden_size=hidden_size,
learning_rate=0.1, iterations=100)

mlp.train(X_train, y_train)

mlp.print_weights()

predictions = mlp.predict(X_test)
accuracy = np.mean(predictions == y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")

```

Input-Hidden Weights:

```
[[ -0.29566566 -0.06310135  0.41898353 -0.05605992]
 [ -0.087182   -0.60413012  0.17030378 -0.5394112 ]
 [ -0.32295963 -0.08301213 -0.37512255 -0.98291623]
 [  0.17780732 -0.84257121  0.14854828 -0.84396401]
 [ -0.15031219 -0.37544889  0.17065824 -1.16808103]
 [ -0.38596397 -0.20979318  0.34222245 -0.69259762]]
```

Hidden-Output Weights:

```
[[ -0.24410555]
 [  1.13980737]
 [ -0.5982828 ]
 [ -0.55066481]]
```

Test Accuracy: 50.00%

SOFT COMPUTING

ASSIGNMENT -7

Perumalla Dharan

AP21110010201

1. Write a Python program to train a Back Propagation Neural Network (BPNN) for classifying whether a student passes or fails using a dataset of students' course marks. Assume necessary parameters.

```
import numpy as np
import pandas as pd

# Load the dataset
train_path = r'E:\SRM\Soft Computing\Lab 7 - 25th
Sept\training_dataset_students(1000).csv'
train_df = pd.read_csv(train_path)
test_path = r'E:\SRM\Soft Computing\Lab 7 - 25th
Sept\students_testing.csv'
test_df = pd.read_csv(test_path)

X_train = train_df[['c1', 'c2', 'c3', 'c4', 'c5', 'c6']].values
y_train = train_df[['result']].values

X_test = test_df[['c1', 'c2', 'c3', 'c4', 'c5', 'c6']].values
y_test = test_df[['result']].values

class MLP:
    def __init__(self, input_size, hidden_size, learning_rate=0.01,
iterations=10000):
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.weights_input_hidden = np.random.uniform(-0.5, 0.5,
(input_size, hidden_size))
```

```

        self.bias_hidden = np.random.uniform(-0.5, 0.5,
(hidden_size,))
        self.weights_hidden_output = np.random.uniform(-0.5, 0.5,
(hidden_size, 1))
        self.bias_output = np.random.uniform(-0.5, 0.5, (1,))

def sigmoid(self, x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(self, x):
    return x * (1 - x)

def predict(self, X):
    hidden_input = np.dot(X, self.weights_input_hidden) +
self.bias_hidden
    hidden_output = self.sigmoid(hidden_input)
    final_input = np.dot(hidden_output,
self.weights_hidden_output) + self.bias_output
    final_output = self.sigmoid(final_input)
    return np.round(final_output)

def train(self, X, y):
    for epoch in range(self.iterations):
        for i in range(len(X)):
            # Forward pass
            hidden_input = np.dot(X[i],
self.weights_input_hidden) + self.bias_hidden
            hidden_output = self.sigmoid(hidden_input)
            final_input = np.dot(hidden_output,
self.weights_hidden_output) + self.bias_output
            final_output = self.sigmoid(final_input)

            #Finding gradients and errors
            output_error = (y[i] - final_output)

```

```

                                output_gradient = output_error *
self.sigmoid_derivative(final_output)

                                hidden_error =
output_gradient.dot(self.weights_hidden_output.T)
                                hidden_gradient = hidden_error *
self.sigmoid_derivative(hidden_output)

    # Update weights and biases
    #updating weights (V) and bias
        self.weights_hidden_output += self.learning_rate *
hidden_output[:, None] * output_gradient
        self.bias_output += self.learning_rate *
output_gradient

    #updating weights (W) and bias
        self.weights_input_hidden += self.learning_rate *
X[i][:, None] * hidden_gradient
        self.bias_hidden += self.learning_rate

# Model configuration
input_size = X_train.shape[1]
hidden_size = 6

mlp = MLP(input_size=input_size, hidden_size=hidden_size,
learning_rate=0.01, iterations=1000)
mlp.train(X_train, y_train)

# Test predictions and accuracy
predictions = mlp.predict(X_test)
accuracy = np.mean(predictions == y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")

```

Test Accuracy: 54.55%

SOFT COMPUTING

ASSIGNMENT -8

Perumalla Dharan

AP21110010201

1.

a). Write a Python program to train a Back Propagation Neural Network (BPNN) for classifying handwritten digits.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

(training_set_images, training_set_labels), (testing_set_images,
testing_set_labels) = mnist.load_data()

training_set_images = training_set_images.astype('float32') / 255.0
testing_set_images = testing_set_images.astype('float32') / 255.0

training_set_images = np.reshape(training_set_images,
(training_set_images.shape[0], 28 * 28))
testing_set_images = np.reshape(testing_set_images,
(testing_set_images.shape[0], 28 * 28))

training_set_labels = to_categorical(training_set_labels,
num_classes=10)
testing_set_labels = to_categorical(testing_set_labels,
num_classes=10)

model = models.Sequential()
model.add(layers.Input(shape=(28 * 28,)))
model.add(layers.Dense(512, activation='relu'))
```

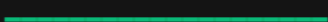
```
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

epochs = 10
for epoch in range(epochs):
    print(f"Running Epoch {epoch + 1} out of {epochs}")
    history = model.fit(training_set_images, training_set_labels,
epochs=1, batch_size=32, validation_split=0.2, verbose=0)
    current_accuracy = history.history['accuracy'][0]
    print(f"Current Accuracy Here: {current_accuracy * 100:.2f}%")

testing_loss, testing_accuracy = model.evaluate(testing_set_images,
testing_set_labels)

print(f"The Accuracy of this BPNN Model Is : {testing_accuracy *
100:.2f}%")
```

```
Running Epoch 1 out of 10
Current Accuracy Here: 93.82%
Running Epoch 2 out of 10
Current Accuracy Here: 97.40%
Running Epoch 3 out of 10
Current Accuracy Here: 98.09%
Running Epoch 4 out of 10
Current Accuracy Here: 98.67%
Running Epoch 5 out of 10
Current Accuracy Here: 98.89%
Running Epoch 6 out of 10
Current Accuracy Here: 99.02%
Running Epoch 7 out of 10
Current Accuracy Here: 99.26%
Running Epoch 8 out of 10
Current Accuracy Here: 99.28%
Running Epoch 9 out of 10
Current Accuracy Here: 99.35%
Running Epoch 10 out of 10
Current Accuracy Here: 99.46%
313/313  0s 1ms/step - accuracy: 0.9725 - loss: 0.1294
The Accuracy of this BPNN Model Is : 97.57%
```

b). Write a Python program to train a Multi Layer Perceptron (MLP) for classifying handwritten digits. Compare the results of both neural networks.

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

def load_data():
    mnist = fetch_openml('mnist_784')
    X, y = mnist['data'], mnist['target'].astype(int)
    return train_test_split(X, y, test_size=0.2, random_state=42)

def preprocess_data(X_train, X_test):
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    return X_train_scaled, X_test_scaled

def train_mlp(X_train, y_train, epochs=10):
    mlp = MLPClassifier(hidden_layer_sizes=(128, 64),
        activation='relu', max_iter=100, warm_start=True, random_state=42)

    for epoch in range(epochs):
        mlp.fit(X_train, y_train)
        yield epoch + 1, mlp

def calculate_accuracy(model, X, y):
    predictions = model.predict(X)
    return np.mean(predictions == y) * 100

X_train, X_test, y_train, y_test = load_data()
X_train_scaled, X_test_scaled = preprocess_data(X_train, X_test)
```

```
epochs = 10
for epoch, model in train_mlp(X_train_scaled, y_train, epochs):
    current_accuracy = calculate_accuracy(model, X_train_scaled,
y_train)
    print(f"Running Epoch {epoch} out of {epochs}")
    print(f"Accuracy in this Epoch Is : {current_accuracy:.2f}%")

final_test_accuracy = calculate_accuracy(model, X_test_scaled,
y_test)

print(f"The Accuracy of this MLP Model Is :
{final_test_accuracy:.2f}%\n")
```

```
Running Epoch 1 out of 10
Accuracy in this Epoch Is : 100.00%
Running Epoch 2 out of 10
Accuracy in this Epoch Is : 99.83%
Running Epoch 3 out of 10
Accuracy in this Epoch Is : 99.82%
Running Epoch 4 out of 10
Accuracy in this Epoch Is : 99.81%
Running Epoch 5 out of 10
Accuracy in this Epoch Is : 99.78%
Running Epoch 6 out of 10
Accuracy in this Epoch Is : 99.86%
Running Epoch 7 out of 10
Accuracy in this Epoch Is : 99.92%
Running Epoch 8 out of 10
Accuracy in this Epoch Is : 99.83%
Running Epoch 9 out of 10
Accuracy in this Epoch Is : 99.90%
Running Epoch 10 out of 10
Accuracy in this Epoch Is : 99.90%
The Accuracy of this MLP Model Is : 97.33%
```

SOFT COMPUTING

ASSIGNMENT -9

Perumalla Dharan

AP21110010201

Given a dataset containing students' marks across multiple courses, develop a KSOM-based model that clusters students based on their academic performance. Use students' marks dataset without considering class label.

```
import numpy as np
import pandas as pd

class SOM:
    def __init__(self, num_features, num_clusters):
        self.weights = np.random.rand(num_features, num_clusters) #
        Initialize weights randomly

    def find_winner(self, sample):
        # Calculate the Euclidean distance from the sample to each weight vector
        distances = np.linalg.norm(self.weights - sample[:, np.newaxis],
axis=0)
        return np.argmin(distances) # Index of the closest weight vector (winning cluster)

    def update_weights(self, sample, winner_index, learning_rate):
        # Update the weights of the winning cluster
        self.weights[:, winner_index] += learning_rate * (sample -
self.weights[:, winner_index])

def load_data(file_path):
    # Load data from CSV file
    data = pd.read_csv(file_path)
    return data
```

```

def main():
    # Load the training dataset
    train_file_path = r'E:\SRM\Soft Computing\Lab 9 - 15th
Oct\training_dataset_students(1000).csv'
    training_data = load_data(train_file_path)

    # Prepare the data: dropping non-numeric and unnecessary columns
    features = training_data.drop(columns=['st_id'],
errors='ignore').values

    num_features = features.shape[1]
    num_clusters = 2 # Two clusters: Pass and Fail

    # Initialize and train the Self-Organizing Map
    som = SOM(num_features, num_clusters)
    learning_rate = 0.6

    # Training process
    for epoch in range(100): # Adjust the number of epochs as necessary
        for sample in features:
            winner_index = som.find_winner(sample)
            som.update_weights(sample, winner_index, learning_rate)
            learning_rate /= 1.01 # Decay the learning rate

    # Testing dataset
    test_file_path = r'E:\SRM\Soft Computing\Lab 9 - 15th
Oct\students_testing.csv'
    testing_data = load_data(test_file_path)
    test_features = testing_data.drop(columns=['st_id'],
errors='ignore').values

    # Classify test samples
    print("Student Cluster Assignments (Pass/Fail) in Testing Dataset:")

```

```

for idx, test_sample in enumerate(test_features):
    winner_index = som.find_winner(test_sample)
    if winner_index == 0:
        result = "Pass" # Cluster 0 for Pass
    else:
        # Check if the student fails based on the given condition
        if np.any(test_sample < 40):
            result = "Fail" # Cluster 1 for Fail
        else:
            result = "Pass"

    print(f"Student {idx + 1} has {result}")

if __name__ == "__main__":
    main()

```

Student Cluster Assignments (Pass/Fail) in Testing Dataset:

```

Student 1 has Fail
Student 2 has Pass
Student 3 has Fail
Student 4 has Fail
Student 5 has Pass
Student 6 has Pass
Student 7 has Fail
Student 8 has Fail
Student 9 has Pass
Student 10 has Pass
Student 11 has Pass

```

SOFT COMPUTING

ASSIGNMENT -10

Perumalla Dharan

AP21110010201

Write a python program to implement different CNNs: AlexNet, LeNet, VGGNet and ResNet for image classification on CIFAR-10 dataset. Compare the results of different CNNs.

```
# Import required libraries
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.applications import VGG16, ResNet50, VGG19
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

# Normalize the data
x_train, x_test = x_train / 255.0, x_test / 255.0

# Convert labels to categorical
y_train, y_test = to_categorical(y_train, 10), to_categorical(y_test, 10)

# Define a simple CNN (LeNet)
def create_lenet():
    model = models.Sequential()
    model.add(layers.Conv2D(6, (5, 5), activation='relu',
input_shape=(32, 32, 3)))
    model.add(layers.AveragePooling2D(pool_size=(2, 2))) # Added
pool_size argument
    model.add(layers.Conv2D(16, (5, 5), activation='relu'))
```

```

    model.add(layers.AveragePooling2D(pool_size=(2, 2))) # Added
pool_size argument
    model.add(layers.Flatten())
    model.add(layers.Dense(120, activation='relu'))
    model.add(layers.Dense(84, activation='relu'))
    model.add(layers.Dense(10, activation='softmax'))
    return model

# Optimized AlexNet for CIFAR-10 (Smaller and Faster)
def create_alexnet():
    model = models.Sequential()
    model.add(layers.Conv2D(64, (3, 3), activation='relu',
input_shape=(32, 32, 3), padding='same'))
    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Conv2D(128, (3, 3), activation='relu',
padding='same'))
    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Conv2D(256, (3, 3), activation='relu',
padding='same'))
    model.add(layers.Conv2D(256, (3, 3), activation='relu',
padding='same'))
    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Flatten())
    model.add(layers.Dense(1024, activation='relu')) # Reduced number of
neurons
    model.add(layers.Dense(512, activation='relu')) # Reduced number of
neurons
    model.add(layers.Dense(10, activation='softmax'))
    return model

```

```

# Use pre-trained VGG16
def create_vgg16():
    base_model = VGG16(weights=None, include_top=False, input_shape=(32,
32, 3))
    model = models.Sequential()
    model.add(base_model)
    model.add(layers.Flatten())
    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.Dense(10, activation='softmax'))
    return model

# Use pre-trained ResNet50
def create_resnet50():
    base_model = ResNet50(weights=None, include_top=False,
input_shape=(32, 32, 3))
    model = models.Sequential()
    model.add(base_model)
    model.add(layers.Flatten())
    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.Dense(10, activation='softmax'))
    return model

# Function to compile and train the model
def compile_and_train(model, epochs=10):
    model.compile(optimizer=Adam(),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    history = model.fit(x_train, y_train, epochs=epochs,
validation_data=(x_test, y_test), batch_size=64)
    return history

# List of models to train
models = {
    'LeNet': create_lenet(),

```

```
'AlexNet': create_alexnet(),
'VGG16': create_vgg16(),
'ResNet50': create_resnet50()
}

# Train each model and display accuracy
results = {}
for model_name, model in models.items():
    print(f"Training {model_name}...")
    history = compile_and_train(model)
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    results[model_name] = test_acc
    print(f"{model_name} Test Accuracy: {test_acc:.4f}")

# Print final results
print("\nComparison of Test Accuracies:")
for model_name, accuracy in results.items():
    print(f"{model_name}: {accuracy:.4f}")
```

Training LeNet...

Epoch 1/10

782/782  **12s** 9ms/step - accuracy: 0.3071 - loss: 1.8805 - val_accuracy: 0.4596 - val_loss: 1.4704

Epoch 2/10

782/782  **7s** 9ms/step - accuracy: 0.4759 - loss: 1.4520 - val_accuracy: 0.5046 - val_loss: 1.3615

Epoch 3/10

782/782  **6s** 8ms/step - accuracy: 0.5231 - loss: 1.3377 - val_accuracy: 0.5311 - val_loss: 1.3058

Epoch 4/10

782/782  **7s** 9ms/step - accuracy: 0.5555 - loss: 1.2443 - val_accuracy: 0.5496 - val_loss: 1.2692

Epoch 5/10

782/782  **7s** 9ms/step - accuracy: 0.5820 - loss: 1.1803 - val_accuracy: 0.5542 - val_loss: 1.2366

Epoch 6/10

782/782  **7s** 8ms/step - accuracy: 0.5963 - loss: 1.1314 - val_accuracy: 0.5853 - val_loss: 1.1654

Epoch 7/10

782/782  **8s** 10ms/step - accuracy: 0.6193 - loss: 1.0798 - val_accuracy: 0.5991 - val_loss: 1.1431

Epoch 8/10

782/782  **7s** 9ms/step - accuracy: 0.6350 - loss: 1.0348 - val_accuracy: 0.5894 - val_loss: 1.1606

Epoch 9/10

782/782  **7s** 9ms/step - accuracy: 0.6455 - loss: 1.0029 - val_accuracy: 0.6000 - val_loss: 1.1431

Epoch 10/10

782/782  **8s** 10ms/step - accuracy: 0.6556 - loss: 0.9735 - val_accuracy: 0.6053 - val_loss: 1.1234

313/313 - 1s - 4ms/step - accuracy: 0.6053 - loss: 1.1234

LeNet Test Accuracy: 0.6053

Training AlexNet...

Epoch 1/10

782/782  **144s** 180ms/step - accuracy: 0.3423 - loss: 1.7638 - val_accuracy: 0.5435 - val_loss: 1.3328

Epoch 2/10

782/782  **145s** 186ms/step - accuracy: 0.6344 - loss: 1.0255 - val_accuracy: 0.6741 - val_loss: 0.9328

Epoch 3/10

782/782  **164s** 210ms/step - accuracy: 0.7269 - loss: 0.7780 - val_accuracy: 0.7105 - val_loss: 0.8339

Epoch 4/10

782/782  **176s** 225ms/step - accuracy: 0.7868 - loss: 0.6140 - val_accuracy: 0.7361 - val_loss: 0.7816

Epoch 5/10

782/782  **195s** 250ms/step - accuracy: 0.8352 - loss: 0.4662 - val_accuracy: 0.7493 - val_loss: 0.7318

Epoch 6/10

782/782  **202s** 258ms/step - accuracy: 0.8784 - loss: 0.3456 - val_accuracy: 0.7566 - val_loss: 0.8470

Epoch 7/10

782/782  **197s** 252ms/step - accuracy: 0.9118 - loss: 0.2510 - val_accuracy: 0.7354 - val_loss: 1.0573

Epoch 8/10

782/782  **200s** 256ms/step - accuracy: 0.9333 - loss: 0.1917 - val_accuracy: 0.7365 - val_loss: 1.0624

Epoch 9/10

782/782  **195s** 250ms/step - accuracy: 0.9485 - loss: 0.1482 - val_accuracy: 0.7326 - val_loss: 1.1262

Epoch 10/10

782/782  **185s** 237ms/step - accuracy: 0.9604 - loss: 0.1191 - val_accuracy: 0.7430 - val_loss: 1.2617

313/313 - 8s - 25ms/step - accuracy: 0.7430 - loss: 1.2617

AlexNet Test Accuracy: 0.7430

SOFT COMPUTING

ASSIGNMENT -11

Perumalla Dharan

AP21110010201

```
import numpy as np

from tensorflow.keras.datasets import mnist

# Loading and normalizing MNIST Dataset

def fetch_mnist():

    (train_X, train_y), (test_X, test_y) = mnist.load_data()

    # Scaling pixel values to the range [0, 1]

    train_X = train_X.astype(np.float32) / 255.0

    test_X = test_X.astype(np.float32) / 255.0

    return (train_X, train_y), (test_X, test_y)

# ReLU activation function for handling non-linearity

def relu_activation(matrix):

    return np.where(matrix > 0, matrix, 0)

# Softmax activation for class probabilities

def softmax_activation(matrix):
```

```

    shift = matrix - np.max(matrix)

    exps = np.exp(shift)

    return exps / np.sum(exps)

# Performing Convolution operation

def convolution(input_mat, filter_mat, stride=1, pad=0):

    # Adding padding to input

    input_padded = np.pad(input_mat, ((pad, pad), (pad, pad)),
mode='constant')

    input_size, filter_size = input_padded.shape[0],
filter_mat.shape[0]

    output_dim = (input_size - filter_size) // stride + 1

    conv_output = np.zeros((output_dim, output_dim))

    for x in range(0, output_dim):

        for y in range(0, output_dim):

            # Selecting a region of the image that matches the size of
filter

            region = input_padded[x*stride:x*stride+filter_size,
y*stride:y*stride+filter_size]

            # Performing element-wise multiplication between the
region and filter

            conv_output[x, y] = np.sum(region * filter_mat)

    return conv_output

```

```

# Performing Max-pooling Operation

def maxpool(input_mat, pool_size=2, stride=2):

    pool_output_dim = (input_mat.shape[0] - pool_size) // stride + 1

    pooled_output = np.zeros((pool_output_dim, pool_output_dim))

    for x in range(0, pool_output_dim):
        for y in range(0, pool_output_dim):
            # Selecting a pool_size x pool_size region

            pool_region = input_mat[x*stride:x*stride+pool_size,
y*stride:y*stride+pool_size]

            # Choosing the max value from the region

            pooled_output[x, y] = np.max(pool_region)

    return pooled_output

# Fully Connected Layer

def dense_layer(flat_input, weights, bias):

    return np.dot(weights, flat_input) + bias

# Defining CNN architecture

def cnn_model(input_img, conv_filters, dense_weights, dense_bias):

    # First convolution Layer with ReLU

```

```
conv1_output = convolution(input_img, conv_filters[0])

relu_output = relu_activation(conv1_output)

# First pooling layer

pooled_output = maxpool(relu_output)

# Flattening the Output for Fully Connected Layer

flattened_vector = pooled_output.flatten()

# Fully Connected Layer

dense_output = dense_layer(flattened_vector, dense_weights,
dense_bias)

# Softmax for Final Classification Probabilities

return softmax_activation(dense_output)

# Loading the Dataset

(train_images, train_labels), (test_images, test_labels) =
fetch_mnist()

# Defining sample filters, weights, and bias

conv_filters = [np.random.randn(3, 3)]

dense_weights = np.random.randn(10, 13 * 13)
```

```
dense_bias = np.random.randn(10)

# Model Testing

example_img = train_images[7]

# Forward passing through the CNN Model

predicted_output = cnn_model(example_img, conv_filters, dense_weights,
dense_bias)

# Displaying the Output Probabilities

print("Predicted Probabilities of The Model Are as Follows:")

for i in range(len(predicted_output)):

    print(f"Probability Image is of Class {i} Is:
{predicted_output[i]:.6f}")
```

Predicted Probabilities of The Model Are as Follows:

Probability Image is of Class 0 Is: 0.866019

Probability Image is of Class 1 Is: 0.000163

Probability Image is of Class 2 Is: 0.016885

Probability Image is of Class 3 Is: 0.011590

Probability Image is of Class 4 Is: 0.000001

Probability Image is of Class 5 Is: 0.000001

Probability Image is of Class 6 Is: 0.000005

Probability Image is of Class 7 Is: 0.000000

Probability Image is of Class 8 Is: 0.000013

Probability Image is of Class 9 Is: 0.105323